PHASE

6

Paging and the Higher-Half Kernel

Turn on the MMU, move the kernel to 0xC0000000, and build the virtual memory layer every later phase stands on

Overall progress toward Turk-OS 1.0 (this phase in red) 34 % → 42 %

Duration	2–3 weeks
Difficulty	★★★★★ (5/5)
You need	Phase 5: the PMM and its tests; confident GDB and QEMU monitor use
You will have	Paging enabled, the kernel linked and running at `0xC0000000`, a direct map of the first 512 MiB of physical memory, a VMM API (`vmm_map`, `vmm_unmap`, `vmm_virt_to_phys`), pre-allocated kernel page tables, and a page-fault handler that explains every fault.

1 Why this phase

Paging is the mechanism behind almost everything a modern OS promises. It lets every process have its own private address space starting at zero (Phase 10), protects the kernel from user programs, shares one physical copy of memory between processes and copies it only when written (Phase 11), and lets the kernel find frames anywhere in RAM and make them look contiguous. This phase turns the MMU on and builds the kernel's side of it. You will not have user processes yet, but every design choice here decides how easy Phases 7–11 will be.

It is also the phase where most hobby kernels stall. When paging is misconfigured, the CPU can't fetch the next instruction, which faults, and the fault handler can't be fetched either, so the machine resets with no message. Work in small steps, check each one in the QEMU monitor before taking the next, and commit after every step that works.

2 Concepts you need

2.1 From address translation to paging

OSTEP builds up to paging in stages, and it is worth following them in order. With **base and bounds** (chapter 15) the hardware adds a base to every address and checks a limit; it is simple but wastes the memory between a program's heap and stack. **Segmentation** (chapter 16, and your Phase 3 GDT) uses several base/bound pairs, but variable-sized segments fragment physical memory. **Paging** (chapter 18) cuts both virtual and physical memory into fixed 4 KiB pieces and keeps a table per address space that maps each virtual page to a physical frame. Fixed sizes mean no external fragmentation, and each page can have its own permissions.

2.2 Two-level paging on x86

A flat table with an entry for every 4 KiB page of a 4 GiB space would take 4 MiB per process. x86 uses two levels instead (Figure 1), which OSTEP chapter 20 explains as a way to avoid storing entries for unused regions. CR3 holds the physical address of a **page directory** of 1024 entries. Each present entry (PDE) points to a **page table** of 1024 entries (PTEs), and each present PTE points to a 4 KiB frame. A 32-bit virtual address is split into a 10-bit directory index, a 10-bit table index and a 12-bit offset. With the PSE extension (CR4 bit 4), a PDE can instead map a whole 4 MiB page directly, with no page table.

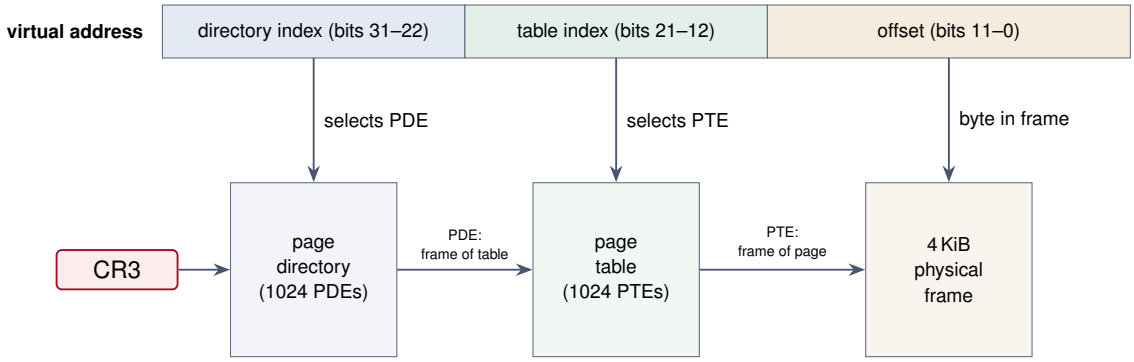


Figure 1: How the MMU translates a 32-bit virtual address with 4 KiB pages. Every pointer in these tables is a *physical* address.

PDEs and PTEs share one format (Figure 2). The top 20 bits are the frame number and the low 12 bits are flags. The three you set most often are **P** (present), **R/W** (writable) and **U/S** (accessible from ring 3). For user access, U/S must be set in both the PDE and the PTE. Bits 9–11 are free for the operating system; Phase 11 uses one of them to mark copy-on-write pages. By default ring 0 may write even to read-only pages; setting **CR0.WP** (bit 16) makes the kernel obey R/W too, which copy-on-write will need.

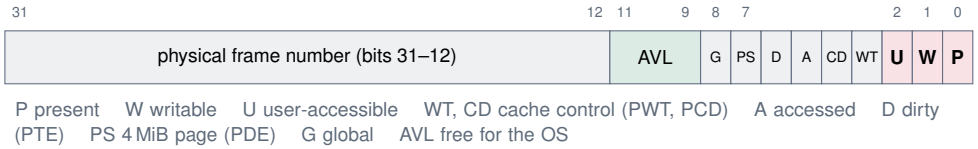


Figure 2: The format of a page directory or page table entry.

2.3 The TLB

Walking two tables in memory for every access would be far too slow, so the CPU caches recent translations in the **TLB** (OSTEP chapter 19). The TLB is *not* kept coherent with your tables automatically. Whenever you change or remove an existing mapping, you must flush it: `invlpg [addr]` for one page, or reload CR3 to flush everything that is not marked global. Forgetting this is the classic source of heisenbugs: the old mapping keeps working for a while, and then it doesn't.

2.4 Why a higher-half kernel

Once user processes exist, each gets its own page directory, but the kernel must be reachable from all of them: interrupts and system calls arrive while a user address space is active. The standard solution is to map the kernel into the top of *every* address space and give the bottom to user programs. Turk-OS uses the Linux-style split: 0–3 GiB for users and 3–4 GiB for the kernel (Figure 3). The kernel is linked to run at `0xC0100000` but still loaded by GRUB at physical `0x00100000`; LOB chapter 9 explains why not to simply identity-map the kernel low instead.

2.5 Reaching physical memory once paging is on

After paging is enabled, the kernel can only touch memory that is mapped. Yet it constantly needs to write to frames it has just taken from the PMM: to zero a new page table, to copy a page, to load a program. LOB chapter 10 lists the options. You can **map frames temporarily** into a reserved slot each time. You can use a **recursive mapping**, where the last PDE points at the directory itself, so the tables appear at `0xFFC00000`. Or you can keep a **direct map** of physical memory in the kernel half, so physical address p is always visible at $p + \texttt{0xC0000000}$. Turk-OS uses the direct map, like xv6

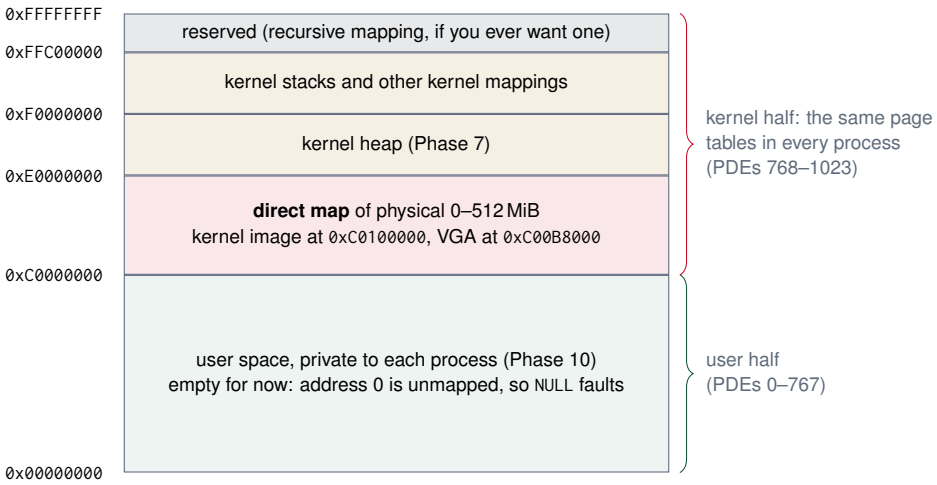


Figure 3: The Turk-OS virtual address space.

and 32-bit Linux: it makes every later phase simpler, at the price of limiting usable RAM to 512 MiB, which is plenty for this project. Two macros, `P2V` and `V2P`, convert between the two kinds of address.

3 Reading plan

Priority	Book	What to read	Pages	Why now
REQUIRED	LOB	Ch. 8 *A Short Introduction to Virtual Memory*; Ch. 9 *Paging* (identity paging, enabling paging, higher-half linker script, entering and running in the higher half)	51–59	The practical route for Tasks 6.1–6.4.
REQUIRED	OSTEP	Ch. 15 *Mechanism: Address Translation*	135–148	What translation is, before you build it.
REQUIRED	OSTEP	Ch. 18 *Paging: Introduction*; Ch. 19 *Paging: Faster Translations (TLBs)*	179–210	Page tables, PTE bits, why the TLB exists and when to flush it.
REQUIRED	OSTEP	Ch. 20 *Paging: Smaller Tables*	211–226	Multi-level tables: exactly the x86 design.
RECOMMENDED	OSC	§9.3 *Paging*; §9.4 *Structure of the Page Table*	360–376	Second explanation with worked examples.
RECOMMENDED	OSC	§9.6 *Example: Intel 32- and 64-bit Architectures*	379–383	The real IA-32 paging hardware, including PAE.
RECOMMENDED	MOS	§3.3 *Virtual Memory* (paging, page tables, speeding up paging, large memories)	194–209	The same ground with more hardware detail.
OPTIONAL	OSDI	§4.3 *Virtual Memory*	383–396	Another careful walk through page tables and TLBs.
OPTIONAL	OSTEP	Ch. 23 *The VAX/VMS Virtual Memory System* (now *Complete Virtual Memory Systems*, §23.1)	255–264	A real system that also maps the kernel into every address space.
RECOMMENDED	LKP	Ch. 7, *Understanding the VM split* and *Examining the kernel segment*	314–326, 340–361	The same 3 GiB/1 GiB split you build, in a production kernel.

OPTIONAL	LKP	Ch. 7, *Randomizing the memory layout – KASLR*	362–367	A hardening idea for Phase 15.

Keep the Intel SDM Volume 3A chapter on paging open, specifically the part on 32-bit paging: it has the exact PDE and PTE formats, the CR0/CR3/CR4 bits, and the page-fault error code.

Real-system lens: how MINIX 3 does it

Here MINIX is a contrast rather than a model. The MINIX 3 version in OSDI does not use paging: each process gets contiguous text and data segments described by its own local descriptor table, and protection comes from segment limits (OSDI §4.6.2 and §4.7.1). Later MINIX releases added paging, for the reasons MOS §3.7.3 gives for why x86 systems abandoned segmentation. If you want a small paging kernel to compare with, look at the MIT xv6 source (`vm.c`), which uses the same direct-map approach as Turk-OS.

Linux, for comparison. LKP 32-bit Linux on x86 uses the same 3 GiB/1 GiB split by default, with low memory mapped directly at `0xC0000000`. Modern kernels also randomise where the kernel sits (KASLR), so an attacker cannot rely on fixed addresses.

4 Build it

Task 6.1 Warm-up: identity paging from C

Before moving anything, prove you can turn paging on without changing any address. In C, take one frame for a page directory and one for a page table, map the first 4 MiB to itself (virtual = physical) with 4 KiB pages, load CR3, and set CR0.PG. If the kernel keeps running, your table format is right. You will throw this code away in Task 6.3, but it isolates the "is my PTE format correct?" question from everything else.

kernel/arch/i386/paging.c (warm-up only)

```
static inline void load_cr3(uintptr_t pd_phys) { __asm__ volatile ("mov %0, %%cr3" : : "r"(pd_phys) : "
    ↪ memory"); }

static inline void enable_paging(void)
{
    uint32_t cr0;
    __asm__ volatile ("mov %%cr0, %0" : "=r"(cr0));
    cr0 |= 0x80000000u;                    /* PG: paging on */
    __asm__ volatile ("mov %0, %%cr0" : : "r"(cr0) : "memory");
}
```

Verify: After `enable_paging()`, `kprintf` still works, and the QEMU monitor's `info mem` shows one range `00000000-00400000` marked `-rw`.

Task 6.2 A page-fault handler that explains itself

Register a handler for vector 14 through `register_interrupt_handler`. Read CR2 and decode the error code: bit 0 tells a non-present page from a protection violation, bit 1 a write from a read, bit 2 user mode from kernel mode, bit 3 a reserved-bit error in a table entry. For now every page fault is fatal: print the decoded message and panic. From Phase 10 on, faults from user mode will kill only the process, and Phase 11 will fix some faults instead of reporting them.

kernel/mm/fault.c

```
static void page_fault_handler(struct regs *r)
{
```

```
    uint32_t addr;
    __asm__ volatile ("mov %%cr2, %0" : "=r"(addr));
    kprintf("Page fault at %08lx: %s, %s, %s mode%s  (EIP=%08lx)\n", addr,
            (r->err_code & 1) ? "protection violation" : "page not present",
            (r->err_code & 2) ? "write" : "read",
            (r->err_code & 4) ? "user" : "kernel",
            (r->err_code & 8) ? ", reserved bit set" : "", r->eip);
    panic("unhandled page fault");
}
```

Verify: With identity paging on, reading `*(volatile int *)0x00800000` (beyond the mapped 4 MiB) prints "page not present, read, kernel mode" with that address.

Task 6.3 Move the kernel to the higher half

Now the real change. Link the kernel at `0xC0100000` but tell the linker to place each section at a load address 3 GiB lower, so GRUB still loads it at 1 MiB. The entry point must be a physical address, because paging is off when GRUB jumps to it.

link.ld

```
ENTRY(_start)
KERNEL_VMA = 0xC0000000;

SECTIONS
{
    . = 0xC0100000;                                  /* virtual: 3 GiB + 1 MiB */
    kernel_start = .;
    .text   ALIGN(4K) : AT(ADDR(.text)   - KERNEL_VMA) { *(.multiboot) *(.text*) }
    .rodata ALIGN(4K) : AT(ADDR(.rodata) - KERNEL_VMA) { *(.rodata*) }
    .data   ALIGN(4K) : AT(ADDR(.data)   - KERNEL_VMA) { *(.data*) }
    .bss    ALIGN(4K) : AT(ADDR(.bss)    - KERNEL_VMA) { *(COMMON) *(.bss*) }
    kernel_end = .;
}
```

The loader builds a boot page directory with 4 MiB pages. It maps the first 4 MiB twice: at 0 (so the instructions right after enabling paging can still be fetched) and at `0xC0000000`. The rest of the direct map (up to 512 MiB) goes in at the same time, so every frame the PMM hands out is reachable from the first moment. After jumping to the higher half, the loader removes the identity mapping.

boot/loader.s (paging part)

```
KERNEL_VMA      equ 0xC0000000
KERNEL_PDE      equ KERNEL_VMA >> 22        ; = 768
DIRECT_MAP_PDES equ 128                     ; 128 x 4 MiB = 512 MiB
PDE_4M_RW       equ 0x83                    ; present | writable | 4 MiB page

global _start
_start equ (loader - KERNEL_VMA)            ; physical entry address for GRUB/QEMU

section .data
align 4096
global boot_page_directory
boot_page_directory:
    dd PDE_4M_RW                            ; PDE 0: identity map 0-4 MiB (temporary)
    times (KERNEL_PDE - 1) dd 0             ; PDEs 1-767: user space, empty
%assign i 0
%rep DIRECT_MAP_PDES                        ; PDEs 768-895: 0xC0000000 + p -> p
    dd (i << 22) | PDE_4M_RW
%assign i i+1
%endrep
    times (1024 - KERNEL_PDE - DIRECT_MAP_PDES) dd 0
```

```
section .text
loader:                                 ; paging is OFF: we run at physical addresses
    mov  ecx, (boot_page_directory - KERNEL_VMA)
    mov  cr3, ecx                       ; CR3 needs the PHYSICAL address
    mov  ecx, cr4
    or   ecx, 0x00000010                ; CR4.PSE: allow 4 MiB pages
    mov  cr4, ecx
    mov  ecx, cr0
    or   ecx, 0x80010000                ; CR0.PG (paging) and CR0.WP (write protect)
    mov  cr0, ecx
    lea  ecx, [higher_half]             ; an absolute address above 0xC0000000
    jmp  ecx

higher_half:                            ; from here on we run in the higher half
    mov  dword [boot_page_directory], 0 ; drop the identity mapping of 0-4 MiB
    invlpg [0]
    mov  esp, kernel_stack_top          ; a virtual address now
    add  ebx, KERNEL_VMA                ; Multiboot info pointer: physical -> virtual
    push ebx
    push eax
    call kmain
```

Verify: The kernel boots as before. In the QEMU monitor, `info mem` shows `c0000000-e0000000` and nothing below it, and `info registers` shows EIP above `0xc0100000`.

Task 6.4 Fix every address that assumed physical = virtual

Add `P2V` and `V2P` to a header, then go through the kernel and fix each place that used a physical address directly. The VGA buffer is now at `P2V(0xB8000)`. The `mmap_addr`, `mods_addr` and command line fields of the Multiboot information are physical, so read them through `P2V`. The PMM receives `V2P(kernel_start)` and `V2P(kernel_end)` when reserving the kernel image, stores its bitmap at a virtual address, and ignores any RAM above 512 MiB.

include/mm.h

```
#define KERNEL_VMA      0xC0000000u
#define DIRECT_MAP_SIZE (512u * 1024 * 1024)

/* Valid only for physical addresses below DIRECT_MAP_SIZE. */
#define P2V(pa) ((void *)((uintptr_t)(pa) + KERNEL_VMA))
#define V2P(va) ((uintptr_t)(va) - KERNEL_VMA)
```

Verify: All Phase 5 tests pass again. `mem` reports at most 512 MiB even with `-m 1G`.

Task 6.5 The VMM: mapping 4 KiB pages

Write `kernel/mm/vmm.c`. The core is a function that finds, and optionally creates, the PTE for a virtual address in a given page directory. On top of it, `vmm_map` installs a mapping, `vmm_unmap` removes one and flushes the TLB, and `vmm_virt_to_phys` walks the tables. Every function takes the page directory as a parameter, because in Phase 10 you will use the same code on user address spaces.

kernel/mm/vmm.c

```
#define PG_PRESENT 0x001
#define PG_WRITE   0x002
#define PG_USER    0x004
#define PG_4MB     0x080
typedef uint32_t pde_t, pte_t;

static pte_t *get_pte(pde_t *pd, uintptr_t va, bool create, uint32_t flags)
```

```
{
    pde_t *pde = &pd[va >> 22];
    if (!(*pde & PG_PRESENT)) {
        if (!create)
            return NULL;
        uintptr_t pt = pmm_alloc_frame();
        if (!pt)
            return NULL;
        memset(P2V(pt), 0, PAGE_SIZE);                  /* direct map makes this easy */
        *pde = pt | PG_PRESENT | PG_WRITE | (flags & PG_USER);
    }
    ASSERT(!(*pde & PG_4MB));                            /* not inside a 4 MiB page */
    pte_t *table = P2V(*pde & ~0xFFFu);
    return &table[(va >> 12) & 0x3FF];
}

void vmm_map(pde_t *pd, uintptr_t va, uintptr_t pa, uint32_t flags)
{
    pte_t *pte = get_pte(pd, va, true, flags);
    ASSERT(pte != NULL && !(*pte & PG_PRESENT));          /* never remap silently */
    *pte = (pa & ~0xFFFu) | (flags & 0xFFF) | PG_PRESENT;
    __asm__ volatile ("invlpg (%0)" : : "r"(va) : "memory");
}
```

Verify: Map a fresh frame at `0xF0000000`, write a pattern through it, and read the same pattern through `P2V` of the frame's physical address. After `vmm_unmap`, `vmm_virt_to_phys` returns 0 and an access page-faults with "page not present".

Portability note

Page tables are where architectures differ most. Once this task works, keep everything that knows the x86 format (`pde_t`, `pte_t`, the `PG_` bits, CR3 and `invlpg`) in `kernel/arch/i386/paging.c`, and let the rest of the kernel call only `vmm_map`, `vmm_unmap` and `vmm_virt_to_phys`, with generic flags of your own (`VM_WRITE`, `VM_USER`) that `paging.c` translates into PTE bits. Phase 16 puts these calls behind the architecture interface as `arch_mmu_map` and friends, and the Sv32 tables of Phase 17 and the AArch64 tables of Phase 18 plug in behind them.

Task 6.6 Pre-allocate the kernel's page tables

In Phase 10 every process gets its own page directory, created by copying PDEs 768–1023 from the kernel's. If the kernel later creates a new page table in its own directory (say, because the heap grew into a new 4 MiB region), existing processes would not see it. Avoid the whole problem now: at boot, allocate an empty page table for every PDE from `0xE0000000` to `0xFFBFFFFF` (127 tables, about half a megabyte). After that, kernel PDEs never change, and copying them is always enough.

Verify: `info mem` is unchanged (empty tables map nothing), and a loop over PDEs 896–1022 finds all of them present.

Task 6.7 Test the fault paths deliberately

Add `crash null` (read address 0), `crash kwrite` (write to a page you mapped read-only, which only faults because CR0.WP is set) and `crash unmapped` (read `0xF8000000`) to the monitor, and add VMM tests to the suite: map and unmap many pages, check `vmm_virt_to_phys` after each, and check that the PMM free count returns to its starting value.

Verify: Each crash command gives a correctly decoded page-fault report, and `make test` passes.

Task 6.8 Make the kernel's code read-only

A stray write through a bad pointer can land anywhere in the kernel, and the worst place is its own code: the damage shows up much later, in a function that is fine. With CR0.WP already set, marking `.text` and `.rodata` read-only turns that silent corruption into an immediate page fault that names the address. There is one obstacle. The loader maps the kernel image with writable 4 MiB pages, and a 4 MiB page has one R/W bit for all of it; that is why `get_pte` asserts it is never used inside one. So first replace each 4 MiB PDE that covers the kernel image with a 4 KiB page table that maps exactly the same frames, then clear R/W on the pages from `kernel_start` up to a new linker-script symbol, `kernel_ro_end`.

link.ld (one line added after .rodata)

```
    .rodata ALIGN(4K) : AT(ADDR(.rodata) - KERNEL_VMA) { *(.rodata*) }
    kernel_ro_end = ALIGN(4K);                          /* .text and .rodata end here */
    .data   ALIGN(4K) : AT(ADDR(.data)   - KERNEL_VMA) { *(.data*) }
```

kernel/mm/vmm.c (protecting the kernel image)

```
extern char kernel_start[], kernel_ro_end[], kernel_end[];

/* Split the 4 MiB pages holding the kernel image into 4 KiB pages, then make
   .text and .rodata read-only. Called once in kmain, right after pmm_init. */
void vmm_protect_kernel(pde_t *pd)
{
    for (uintptr_t va = (uintptr_t)kernel_start & ~0x3FFFFFu;
         va < (uintptr_t)kernel_end; va += 0x400000) {
        pde_t *pde = &pd[va >> 22];
        ASSERT(*pde & PG_4MB);
        uintptr_t pt = pmm_alloc_frame();
        ASSERT(pt != 0);                            /* boot time: no memory is a bug */
        pte_t *table = P2V(pt);
        uintptr_t frame = *pde & 0xFFC00000u;       /* first frame of the 4 MiB page */
        for (uint32_t i = 0; i < 1024; i++)         /* same mapping, finer grain     */
            table[i] = (frame + i * PAGE_SIZE) | PG_PRESENT | PG_WRITE;
        *pde = pt | PG_PRESENT | PG_WRITE;
    }
    for (uintptr_t va = (uintptr_t)kernel_start; va < (uintptr_t)kernel_ro_end; va += PAGE_SIZE)
        *get_pte(pd, va, false, 0) &= ~(pte_t)PG_WRITE;
    __asm__ volatile ("mov %0, %%cr3" : : "r"(V2P(pd)) : "memory");   /* flush every old entry */
}
```

Call it with `boot_page_directory`. Then add `crash ktext` to the monitor: it writes one byte to the first instruction of `kmain`, through `*(volatile uint8_t *)(uintptr_t)kmain = 0xC3`.

One limit remains, and you should write it down rather than discover it later. Legacy 32-bit paging has no NX bit: every page that can be read can also be executed. Read-only code stops writes to code, but nothing stops the CPU from executing bytes in a data page, so the i386 build cannot promise that writable memory is never executed. Add that line to the out-of-scope list in `docs/security/threat-model.md`. PAE paging has an NX bit; the RISC-V and ARM ports of Phases 17 and 18 have one too.

Verify: `crash ktext` prints "protection violation, write, kernel mode" with error code 3 (present, write) and an address inside `kmain`. `info mem` shows the range from `c0100000` to `kernel_ro_end` as `-r-` and the rest of the kernel as `-rw`, and all PMM and VMM tests still pass.

5 Testing and debugging

The QEMU monitor is your best friend in this phase. `info mem` lists mapped virtual ranges with their permissions, `info tlb` lists individual page mappings, and `info registers` shows CR0, CR2, CR3 and CR4. Run with `-d int,cpu_reset -no-reboot` so a triple fault stops QEMU and shows the last exceptions instead of rebooting.

Symptom	Likely cause	What to do
Triple fault the instant `CR0.PG` is set	The next instruction isn't mapped (no identity mapping), or CR3 got a virtual address	Keep PDE 0 until after the jump; load CR3 with `boot_page_directory - KERNEL_VMA`.
Triple fault right after the jump to `higher_half`	The linker script VMA/LMA split is wrong, or PSE wasn't enabled so `0x83` entries were read as table pointers	Check `i686-elf-readelf -l`: VirtAddr `0xC01...`, PhysAddr `0x001...`; check CR4 bit 4.
GRUB or QEMU refuses the kernel after the move	The entry point is a virtual address	Use the physical `_start` symbol as `ENTRY`.
Page fault inside the page-fault handler, then reset	The handler, its stack or `kprintf`'s data touches unmapped memory	Everything the fault path uses must be in the kernel half, which is always mapped.
Old data appears after changing a mapping, or a fault after unmapping arrives late or never	Stale TLB entry	`invlpg` after every change to an existing entry.
Garbage when reading Multiboot data or boot modules	Physical addresses used as pointers	Read them through `P2V`; make sure they lie below 512 MiB.
Triple fault or random crash right after splitting the kernel PDE	The new page table maps the wrong frames, or the old 4 MiB entry is still in the TLB	Fill all 1024 entries from the PDE's base (`*pde & 0xFFC00000`) before installing it, then reload CR3.
`get_pte` assertion "not inside a 4 MiB page" when clearing R/W	The PDE covering that part of the image was never split	Split every PDE from `kernel_start` rounded down to 4 MiB up to `kernel_end`.
Page fault with error code 3 on `lgdt`, a segment load or `ltr`	The GDT or TSS is `const` and sits in `.rodata`; the CPU writes the accessed and busy bits	Keep descriptor tables in `.data` or `.bss`.

6 Milestone

Milestone: Virtual memory is on

- ☐ The kernel runs at `0xC0100000`; `info mem` shows no mapping below `0xC0000000`.
- ☐ The direct map covers 512 MiB, and `P2V/V2P` are used everywhere a physical address is needed.
- ☐ `vmm_map`, `vmm_unmap` and `vmm_virt_to_phys` work on any page directory and flush the TLB.
- ☐ Kernel page tables for `0xE0000000`–`0xFFBFFFFF` are pre-allocated.
- ☐ A null-pointer read, a write to a read-only page and an unmapped access each produce a clear report.
- ☐ All PMM and VMM tests pass.
- ☐ The kernel image is mapped with a 4 KiB page table, `.text` and `.rodata` are read-only, and `crash ktext` page-faults with error code 3 from ring 0.
- ☐ The threat model records that 32-bit paging has no NX bit.

Self-check questions

1. Translate `0xC0101234` by hand: which PDE index, which PTE index, which offset? With the Turk-OS boot directory, which physical address does it reach?
2. Why must the code that enables paging be identity-mapped at that moment?
3. What do CR2 and CR3 hold? Why does CR3 need a physical address?
4. When exactly must you execute `invlpg`? When is reloading CR3 better?
5. Why do both the PDE and the PTE need the U/S bit for user access?
6. Why put the kernel in the higher half, and why share the same kernel page tables between all processes?
7. Compare a direct map, a recursive mapping and temporary mappings. What does each cost?

7 Going further

Mark kernel mappings *global* (PTE bit 8, enabled by CR4.PGE) so they survive CR3 reloads, which makes the context switches of Phase 8 cheaper. If you want NX protection (no execution from data pages), read about PAE paging, which has three levels and 64-bit entries.

Next: **Phase 7 Kernel Heap and Core Data Structures** →

Books used in this plan

LOB Erik Helin, Adam Renberg. *The little book about OS development*, 2015. The hands-on x86 guide. Free at `https://littleosbook.github.io`.

OSTEP Remzi H. Arpaci-Dusseau, Andrea C. Arpaci-Dusseau. *Operating Systems: Three Easy Pieces*, v0.90, Arpaci-Dusseau Books, 2015. Main theory text. Free at `https://pages.cs.wisc.edu/~remzi/OSTEP/`.

OSC A. Silberschatz, P. B. Galvin, G. Gagne. *Operating System Concepts*, 10th ed., Wiley, 2018. Reference theory text.

MOS Andrew S. Tanenbaum, Herbert Bos. *Modern Operating Systems*, 4th ed., Pearson, 2015. Hardware, I/O, design, security.

OSDI Andrew S. Tanenbaum, Albert S. Woodhull. *Operating Systems: Design and Implementation*, 3rd ed., Pearson Prentice Hall, 2006. MINIX 3 with full source listing.

LKD Robert Love. *Linux Kernel Development*, 3rd ed., Addison-Wesley, 2010. How Linux implements each subsystem.

LKP Kaiwan N. Billimoria. *Linux Kernel Programming*, Packt, 2021. Modern kernel practice: logging, allocators, the scheduler, synchronisation.

The books do not come with this plan. LOB and OSTEP are free online; buy or borrow the other five (the roadmap lists their ISBNs). Where books disagree, the plan follows the newer one when its method is also better for Turk-OS. Page numbers are the *printed* page numbers of these editions, not PDF page indices. With another edition, or with the free OSTEP chapters (each one starts at page 1), find the reading by its chapter and section numbers. MINIX source references like `kernel/proc.c (07400)` give the file and its starting line in OSDI Appendix B.